MINI-PROJET POO · SUJET 5

Usine intelligente 4.0 : Sahel Assemblage

Cahier des charges pédagogique

Wissem Zeddini · À lire avec le document « Mini-projets POO : cadre commun »

Client	Mme Rym Chaabane, directrice de la production de la Manufacture Sahel Assemblage (client simulé)
Le logiciel	Pilotage d'une usine d'assemblage automatisée : des robots, des bras de soudure et des drones reçoivent un plan de production
Difficulté	Intermédiaire (2 sur 3), avec un défi métier plus exigeant
Notions du socle	Encapsulation, constructeurs, héritage, redéfinition, polymorphisme
Bonus possibles	Abstraction (badge Abstrait), exceptions personnalisées (badge Résilient), sauvegarde dans un fichier (badge Archiviste)
Livrable final	Code, fichier .jar, UML, rapport de 2 à 3 pages, démonstration de 5 minutes

1. Le contexte

La **Manufacture Sahel Assemblage** (une usine fictive) fabrique des équipements électroniques dans un atelier entièrement automatisé : des robots assemblent, des bras soudent, des drones apportent les pièces d'un poste à l'autre. Chaque matin, la direction prépare un **plan de production** : une liste d'ordres à répartir entre les machines.

Or les machines chauffent, s'usent et tombent en panne. **Vous êtes l'équipe de pilotage** : votre logiciel confie les ordres aux machines, suit leur état et ne doit jamais laisser la chaîne s'arrêter. Il n'y a ni écran graphique ni capteur réel : seulement des objets, affichés dans la console. Le client changera d'avis : concevez dès le départ pour que votre code puisse évoluer.

Attention : ce n'est pas l'exercice « Usine et Personnel » du TP 6 : ici, seulement des machines et des ordres.

2. Le vocabulaire du métier

Terme	Sens pour le client
Machine	Un équipement de l'usine qui exécute des ordres : robot d'assemblage, bras de soudure, drone de livraison interne, convoyeur.
Ordre de production	Un travail à faire : un type de tâche et une durée en heures. L'ordre retient les heures déjà faites.
Type de tâche	Ce qu'une machine sait faire : manutention, assemblage, soudure ou transport.
Température, vitesse	La température du moteur (en °C), qui monte quand la machine travaille, et sa vitesse de rotation (en tr/min).
Heures de fonctionnement	Le compteur d'usure d'une machine : il ne fait qu'augmenter.
Statut	L'état d'une machine : opérationnelle, en surchauffe (moteur trop chaud) ou en panne.
Intervalle de révision	Le nombre d'heures de fonctionnement entre deux pannes, propre à chaque type de machine.

Terme	Sens pour le client
Coût horaire	Le prix d'une heure de travail d'une machine, en millimes (1 dinar = 1 000 millimes).
Réaffectation	Confier à une autre machine les heures qu'une machine arrêtée n'a pas faites.

3. Les règles du métier

Ces règles sont celles du client. Chacune doit être **protégée par votre code**, pas seulement respectée par vos tests.

N°	Règle	Apparaît en
R1	La température du moteur reste **toujours entre 20 °C (température ambiante) et 150 °C**.	v0.2
R2	La vitesse de rotation reste **toujours entre 0 et la vitesse maximale** du type de machine.	v0.2
R3	Les heures de fonctionnement **ne diminuent jamais** : on peut seulement en ajouter, un nombre positif à la fois.	v0.2
R4	Une machine est **disponible** si son statut est « opérationnelle ».	v0.2
R5	À partir de **90 °C**, la machine est en **surchauffe** ; elle redevient opérationnelle quand elle est refroidie à **60 °C ou moins**. Seule une réparation fait sortir une machine de la panne.	v0.2
R6	Chaque type de machine a **ses propres valeurs** (tableau ci-dessous). Chaque heure de travail ajoute son échauffement, une heure de fonctionnement et son coût horaire.	v0.3
R7	Une machine **refuse** un ordre qu'elle ne sait pas faire, ou si elle n'est pas disponible : rien n'est consommé et la réponse est « non ».	v0.3
R8	Dans une usine, **le nom d'une machine est unique** et la capacité maximale ne peut pas être dépassée.	v0.3
R9	Pendant un ordre, une machine qui passe en surchauffe ou en panne **s'arrête aussitôt** ; seules les heures faites sont facturées. La panne survient à chaque **multiple de l'intervalle de révision** (en heures de fonctionnement) et l'emporte sur la surchauffe.	v1.0
R10	Un ordre interrompu est **confié automatiquement à une autre machine capable** et disponible, qui ne fait que les heures restantes. S'il n'y en a aucune, l'ordre reste en attente.	v1.0

Les types de machines

Type	Tâches acceptées	Échauffement (°C/h)	Vitesse max (tr/min)	Révision (h)	Coût horaire (millimes)
Machine (classe mère) : un convoyeur	Manutention	10	3000	1000	10000
Robot d'assemblage	Assemblage	12	4000	800	25000
Bras de soudure	Soudure	20	2000	600	40000
Drone de livraison interne	Transport et manutention	15	6000	500	15000

*Utilisez des **constantes nommées** (seuils de température, statuts) plutôt que des nombres écrits au milieu du code. Les températures, les heures et les coûts sont des **entiers** ; pas de nombres à virgule pour l'argent.*

4. Version v0.1 : Cadrage

Mail du client · v0.1

De : Rym Chaabane, Direction de la Production, Manufacture Sahel Assemblage

Objet : Notre future usine pilotée par logiciel

Bonjour,

Nous automatisons notre atelier d'assemblage : des robots, des bras de soudure et des drones de livraison interne. Avant de coder quoi que ce soit, je voudrais être sûre que vous avez compris mon métier. Montrez-moi les objets que vous avez repérés, ce que chacun sait de lui-même, et affichez-moi une première machine à l'écran. Pas d'écran graphique : la console me suffit.

Cordialement.

N°	Demande du client	Notion
1.1	« En tant que directrice de la production, je veux que vous identifiiez les objets de mon usine, afin que nous parlions le même langage. »	Analyse, UML
1.2	« En tant que directrice, je veux voir une machine à l'écran, afin de vérifier que mon besoin est compris. »	Classe, constructeur, main

Vous livrez

- Un **UML de départ** d'au moins 3 classes (par exemple Machine, Ordre et Demo), avec leurs attributs principaux.
- Des **squelettes de classes qui compilent**.
- Une classe **Demo** dont le main affiche une machine, par exemple : `Convoyeur CV-01 · 20 °C · 0 tr/min · 0 h · opérationnelle`.

Indice

Une machine connaît au moins son **nom**, sa **température**, sa **vitesse de rotation**, ses **heures de fonctionnement** et son **statut**. Les types (robot, bras, drone) arrivent avec le mail de la version v0.3 : ne les devancez pas.

5. Version v0.2 : Objets fiables

Mail du client · v0.2

De : Rym Chaabane

Objet : Une machine à 400 °C

Bonjour,

Merci pour votre première version. Mes techniciens l'ont testée : l'un a saisi 400 °C pour le moteur d'un convoyeur, et le logiciel l'a accepté ! Un autre a retiré 10 heures de fonctionnement à une presse pour « rajeunir » son compteur. Mes données doivent rester cohérentes.

Mes règles : la température reste entre 20 et 150 °C, la vitesse ne dépasse jamais le maximum de la machine, les heures ne font qu'augmenter. À partir de 90 °C, une machine est en surchauffe et ne travaille plus ; elle ne repart que refroidie à 60 °C ou moins. Personne ne doit pouvoir modifier la température ou le statut directement.

Je veux aussi créer une machine neuve ou avec son historique, et la lire en une ligne.

Cordialement.

N°	Demande du client	Notion
2.1	« Je veux que la température du moteur reste toujours entre 20 et 150 °C, afin d'éviter des mesures absurdes. » (R1)	Encapsulation, validation
2.2	« Je veux régler la vitesse de rotation sans dépasser le maximum de la machine, afin de protéger les moteurs. » (R2)	Setter qui valide
2.3	« Je veux ajouter des heures de fonctionnement, jamais en retirer, afin de suivre l'usure. » (R3)	Méthode métier
2.4	« Je veux savoir si une machine est disponible, afin de ne lui confier du travail que si elle est en état. » (R4)	Méthode booléenne
2.5	« Je veux chauffer et refroidir une machine, la déclarer en panne et la réparer, le statut suivant la température, afin de refléter la réalité. » (R5)	État d'un objet
2.6	« Je veux créer une machine neuve ou avec son historique (heures, température), et la lire en une ligne, afin de gagner du temps. »	Constructeurs surchargés, toString()

Vous livrez

- Tous les attributs **privés**, et **aucun setter** pour la température, les heures et le statut : seules des méthodes métier (chauffer, refroidir, ajouter des heures, déclarer une panne, réparer) les modifient.
- Deux constructeurs (machine neuve, machine avec historique) qui **valident** les valeurs reçues ; le statut se **déduit** de la température.
- Une **Demo** avec au moins trois machines, qui montre des **refus** : température de 200 °C, heures négatives, vitesse trop élevée.

Indices

Testez les **valeurs limites** de température : 19, 20, 89, 90, 150 et 151. Les méthodes `Math.min` et `Math.max` gardent une valeur entre deux bornes. Le statut ne se saisit jamais : il se déduit, y compris dans le constructeur, qui est la porte d'entrée la plus oubliée. Pas encore d'enum : des constantes entières, par exemple `STATUT_EN_PANNE`, suffisent.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Heures invalides (-10) : 0 utilisée.
Température invalide (200 °C) : 150 °C utilisée.
Convoyeur CV-01 · 20 °C · 0 tr/min · 0 h · opérationnelle
Presse PR-02 · 45 °C · 0 tr/min · 250 h · opérationnelle
Fraiseuse FR-03 · 150 °C · 0 tr/min · 0 h · EN SURCHAUFFE
  Vitesse invalide (4000 tr/min) : elle doit être entre 0 et 3000 tr/min.
  Heures invalides (-2) : il faut ajouter un nombre positif.
Convoyeur CV-01 · 20 °C · 2500 tr/min · 8 h · opérationnelle
Convoyeur CV-01 · 95 °C · 2500 tr/min · 8 h · EN SURCHAUFFE
Convoyeur CV-01 disponible ? false
Convoyeur CV-01 · 75 °C · 2500 tr/min · 8 h · EN SURCHAUFFE
Convoyeur CV-01 · 45 °C · 2500 tr/min · 8 h · opérationnelle
Convoyeur CV-01 disponible ? true
Convoyeur CV-01 · 45 °C · 0 tr/min · 8 h · EN PANNE
  Réparation de Convoyeur CV-01 effectuée.
  Convoyeur CV-01 n'est pas en panne : réparation inutile.
Convoyeur CV-01 · 45 °C · 0 tr/min · 8 h · opérationnelle
```

6. Version v0.3 : La famille

> **Mail du client · v0.3**
>
> **De :** Rym Chaabane
>
> **Objet :** Trois types de machines, et ce n'est pas fini
>
> Bonjour,
>
> Excellent travail, mes techniciens ne trichent plus. Passons à l'usine réelle. J'ai trois types de machines : des **robots d'assemblage**, des **bras de soudure** et des **drones de livraison interne** (voir le tableau). Chacun a sa tâche, chauffe à son rythme, s'use à sa vitesse et coûte son prix à l'heure.
>
> Je veux une **usine** qui regroupe des machines de tous types, qui les affiche d'un coup, et à qui j'envoie un **plan de production** : une liste d'ordres, chacun confié à la première machine capable et disponible. À la fin, je veux savoir ce que le plan a coûté.
>
> Et une chose importante : l'an prochain, j'aurai peut-être des robots de peinture. Je ne veux pas que vous réécriviez votre programme à chaque nouveau type.
>
> Cordialement.

N°	Demande du client	Notion
3.1	« Je veux trois types de machines avec leurs particularités, afin de décrire fidèlement mon atelier. »	Héritage, super(...)
3.2	« Je veux que chaque type ait sa tâche, son échauffement, sa vitesse maximale, sa révision et son coût horaire, afin d'appliquer mes règles. » (R6)	Redéfinition, @Override
3.3	« Je veux une usine qui contient des machines de tous types, avec une capacité maximale et sans doublon de nom, afin de refléter mon atelier. » (R8)	Tableau du type commun
3.4	« Je veux afficher toutes les machines en une fois, afin de contrôler l'usine. »	Une seule boucle, toString()
3.5	« Je veux envoyer un plan de production à l'usine : chaque ordre est confié à la première machine capable et disponible, afin de répartir le travail. » (R6, R7)	Polymorphisme : executerTache
3.6	« Je veux ajouter un nouveau type de machine plus tard sans modifier l'usine, afin que mon logiciel évolue. »	Preuve du polymorphisme

Vous livrez

- Une **classe mère** qui porte ce qui est commun (état, heures, coût, exécution d'un ordre) et **trois classes filles** qui n'ajoutent que leurs particularités. L'UML est mis à jour.
- Une classe **Ordre** (type de tâche, durée en heures) et une **Usine** qui contient des machines de types différents, avec **une seule boucle** pour les afficher et pour confier un ordre.
- La preuve que l'usine n'a pas besoin d'être modifiée pour accueillir un nouveau type.

> **Indices**
>
> Si vous écrivez **instanceof** dans l'usine, c'est que le polymorphisme manque : quelle méthode chaque type devrait-il redéfinir ? Cherchez ce qui change d'un type à l'autre (échauffement, coût, tâches acceptées) et ce qui reste identique (la boucle des heures) : ce qui est identique ne se recopie pas dans les filles. Le drone accepte deux types de tâches : chaque type répond à sa façon à la question « sais-tu faire cette tâche ? ». Pour comparer deux noms (du texte), utilisez `equals`, jamais `==`.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Nom RA-01 déjà présent : ajout refusé.
Usine pleine (5/5) : ajout de BS-02 refusé.
Usine Sahel Assemblage (5/5)
Convoyeur CV-01 · 20 °C · 0 tr/min · 0 h · opérationnelle
Robot d'assemblage RA-01 · 20 °C · 0 tr/min · 0 h · opérationnelle
Bras de soudure BS-01 · 20 °C · 0 tr/min · 0 h · opérationnelle
Drone de livraison interne DR-01 · 20 °C · 0 tr/min · 0 h · opérationnelle
Robot d'assemblage RA-02 · 20 °C · 0 tr/min · 0 h · opérationnelle
Disponibles : 5
RA-01 prend en charge l'ordre 1 : assemblage, il reste 4 h.
  RA-01 termine l'ordre 1 : 4 h, 100000 millimes.
BS-01 prend en charge l'ordre 2 : soudure, il reste 3 h.
  BS-01 termine l'ordre 2 : 3 h, 120000 millimes.
DR-01 prend en charge l'ordre 3 : transport, il reste 4 h.
  DR-01 termine l'ordre 3 : 4 h, 60000 millimes.
Convoyeur CV-01 prend en charge l'ordre 4 : manutention, il reste 2 h.
  Convoyeur CV-01 termine l'ordre 4 : 2 h, 20000 millimes.
Plan : 4 ordre(s) terminé(s) sur 4.
Coût total : 300000 millimes
```

7. Version v1.0 : Livraison

> **Mail du client · v1.0**
>
> **De :** Rym Chaabane
>
> **Objet :** Dernière étape avant la mise en production
>
> Bonjour,
>
> L'usine tourne bien… jusqu'à ce qu'une machine surchauffe en plein ordre. Mon cauchemar, c'est la chaîne arrêtée. Je veux que le logiciel simule le travail **heure par heure** : si une machine passe en surchauffe ou tombe en panne, elle s'arrête sur-le-champ, on ne compte que les heures qu'elle a faites, et **l'ordre est confié automatiquement à une autre machine capable** pour les heures restantes. S'il n'y en a aucune, l'ordre attend.
>
> Une machine tombe en panne à chaque multiple de son intervalle de révision ; seule une réparation la remet en service.
>
> Je veux aussi un menu pour mes superviseurs et un fichier que je lance moi-même.
>
> Cordialement.

N°	Demande du client	Notion
4.1	« En tant que directrice, je veux envoyer un plan complet et en voir le déroulement, afin de tester mes règles avant la mise en production. »	Assemblage, boucle
4.2	« Je veux qu'une machine qui surchauffe s'arrête aussitôt et que l'ordre passe à une autre machine capable, afin que la production ne s'arrête pas. » (R9, R10)	Défi métier, polymorphisme
4.3	« Je veux qu'une machine tombe en panne à chaque multiple de son intervalle de révision, afin de planifier la maintenance. » (R9)	État d'un objet
4.4	« Je veux que seules les heures réellement travaillées soient comptées, en millimes, afin de connaître le vrai coût de production. » (R6, R9)	Calcul en entiers
4.5	« Je veux un menu console, afin que mes superviseurs utilisent le logiciel sans toucher au code. »	Menu, boucle de saisie
4.6	« Je veux un fichier que je lance moi-même, afin de l'installer sur mon poste. »	Fichier .jar

Le défi métier : la chaîne qui ne s'arrête pas

Pendant un ordre, la machine travaille **heure par heure** : après chaque heure, elle chauffe, compte une heure de plus et ajoute son coût. Dès qu'elle n'est plus disponible :

1. elle s'arrête sur-le-champ et indique combien d'heures elle a faites ;

2. seules ces heures sont comptées et facturées ;

3. elle passe en surchauffe (jusqu'à son refroidissement à 60 °C) ou en panne (jusqu'à sa réparation), la panne l'emportant si les deux arrivent à la même heure ;

4. l'usine confie les heures restantes à une autre machine capable et disponible, sans demander « quel est ton type ? » ; s'il n'y en a aucune, l'ordre reste en attente.

Dans le socle, pas d'exceptions : des **états** et des **valeurs de retour booléennes** suffisent.

Menu console attendu (exemple) :

```
=== USINE SAHEL ASSEMBLAGE ===
1. Afficher les machines
2. Confier un ordre de production
3. Envoyer le plan de la journée
4. Refroidir les machines
5. Réparer une machine en panne
6. Coût total de production
0. Quitter
Votre choix (0 à 6) :
```

Exemple de sortie d'un plan de production (la vôtre peut différer). Le drone DR-01 est créé avec 498 heures de fonctionnement, et l'ordre 4 reste en attente :

```
--- Plan de la journée ---
RA-01 prend en charge l'ordre 1 : assemblage, il reste 4 h.
  RA-01 termine l'ordre 1 : 4 h, 100000 millimes.
RA-01 prend en charge l'ordre 2 : assemblage, il reste 5 h.
  Surchauffe de RA-01 (92 °C) : arrêt après 2 h, 50000 millimes. Reste 3 h.
RA-02 prend en charge l'ordre 2 : assemblage, il reste 3 h.
  RA-02 termine l'ordre 2 : 3 h, 75000 millimes.
BS-01 prend en charge l'ordre 3 : soudure, il reste 3 h.
  BS-01 termine l'ordre 3 : 3 h, 120000 millimes.
BS-01 prend en charge l'ordre 4 : soudure, il reste 3 h.
  Surchauffe de BS-01 (100 °C) : arrêt après 1 h, 40000 millimes. Reste 2 h.
  Ordre 4 en attente : aucune machine disponible pour les 2 h restantes.
DR-01 prend en charge l'ordre 5 : manutention, il reste 3 h.
  Panne de DR-01 (50 °C) : arrêt après 2 h, 30000 millimes. Reste 1 h.
Convoyeur CV-01 prend en charge l'ordre 5 : manutention, il reste 1 h.
  Convoyeur CV-01 termine l'ordre 5 : 1 h, 10000 millimes.
RA-02 prend en charge l'ordre 6 : assemblage, il reste 2 h.
  RA-02 termine l'ordre 6 : 2 h, 50000 millimes.
Plan : 5 ordre(s) terminé(s) sur 6.
Usine Sahel Assemblage (5/6)
Robot d'assemblage RA-01 · 92 °C · 0 tr/min · 6 h · EN SURCHAUFFE
Robot d'assemblage RA-02 · 80 °C · 0 tr/min · 5 h · opérationnelle
Bras de soudure BS-01 · 100 °C · 0 tr/min · 4 h · EN SURCHAUFFE
Drone de livraison interne DR-01 · 50 °C · 0 tr/min · 500 h · EN PANNE
Convoyeur CV-01 · 30 °C · 0 tr/min · 1 h · opérationnelle
Coût total : 475000 millimes
```

Indices

La réaffectation se fait dans la **boucle de l'usine** : tant que l'ordre n'est pas terminé, on propose les heures restantes à la machine suivante. L'ordre **mémorise ses heures faites** : ne le recommencez pas à zéro. Calculez les coûts en entiers (heures multipliées par le coût horaire). Lisez bien l'**ordre des vérifications** : une machine indisponible ne doit jamais recevoir un ordre, et un ordre terminé à la dernière heure n'est pas « interrompu », même si la machine surchauffe à cet instant.

8. Les bonus du sujet

Badge	Défi
Abstrait	Rendez la classe mère **abstraite** : une « machine générique » n'existe pas dans l'usine, et le convoyeur devient une classe fille à part entière. Faites de `getTypeTache` et de `getEchauffementParHeure` des méthodes abstraites. Que répond Java si vous écrivez `new Machine(...)` ? Justifiez votre choix dans le rapport.
Résilient	Créez deux **exceptions personnalisées**, `SurchauffeException` et `PanneMachineException`, lancées par la machine qui s'arrête en cours d'ordre et interceptées par l'usine, qui réaffecte alors l'ordre. Comparez avec la version à valeurs de retour booléennes dans votre rapport.
Archiviste	Sauvegardez l'état des machines dans un fichier texte ou CSV à la fermeture (une ligne par machine : type, nom, heures, température, statut, coût accumulé) et rechargez-le au démarrage.

9. Les pièges fréquents de ce sujet

- **Une température hors bornes** : oublier de plafonner à 150 °C, ou de ne pas descendre sous 20 °C après un refroidissement.

- **Un statut qui contredit la température** : un setter de statut, ou une température modifiée sans mettre le statut à jour, laisse une machine « opérationnelle » à 120 °C. Pensez au constructeur et à la réparation.
- **Des heures de fonctionnement qui reculent** ou remises à zéro à la réparation : la panne suivante se calcule sur le total des heures.
- **Un instanceof dans l'usine** : le polymorphisme est contourné ; l'arrivée d'un robot de peinture obligerait à la modifier.
- **Recopier la boucle des heures dans chaque classe fille** : si vous copiez, c'est que la classe mère manque ; chaque fille ne précise que ses valeurs.
- **Repartir de zéro à la réaffectation** : sans mémoire des heures faites, des heures sont refaites et facturées deux fois ; facturer l'ordre entier à la machine tombée en panne est la même erreur.
- **Une boucle sans issue** quand aucune machine n'est disponible : prévoyez la sortie « ordre en attente ».
- **Oublier super(...)** dans le constructeur d'une fille, ou le placer après une autre instruction.
- **Reprendre l'exercice « Usine et Personnel » du TP 6** : ici, ni employés ni salaires.

10. Pour votre rapport

En plus de la structure du cadre commun, répondez à ces questions propres au sujet :

- Pourquoi n'y a-t-il pas de setter pour la température, les heures et le statut ? Quelles règles sont ainsi protégées ?
- Quelle duplication de code la classe mère a-t-elle évitée entre robot, bras et drone ?
- Quelle est la ligne de code où le **type réel** d'une machine décide de son échauffement, de son coût ou des tâches qu'elle accepte ?
- Que faudrait-il modifier pour ajouter un robot de peinture ? Combien de classes existantes ?
- Comment votre code garantit-il qu'une machine en surchauffe ou en panne ne reçoit jamais d'ordre, et qu'aucune heure n'est facturée deux fois ?

11. Modifications en direct possibles à la soutenance

Pendant la démonstration, on pourra vous demander l'une de ces modifications, en quelques minutes :

- Ajouter un **robot de peinture** (nouvelle tâche, 9 °C par heure, 3500 tr/min, révision à 700 h, 20000 millimes l'heure) et le voir travailler sans toucher à l'usine.
- Passer le seuil de surchauffe de 90 °C à 80 °C.
- Passer l'intervalle de révision du drone de 500 à 400 heures.
- Ajouter un statut **en maintenance** qui empêche tout ordre.

Annexe : les liens entre les classes (indicatif)

Voici seulement les **relations** attendues à l'arrivée. Les attributs et les méthodes, c'est à vous de les concevoir.

Relation	Sens
RobotAssemblage, BrasSoudure, DroneLivraisonInterne → Machine	Héritage : chaque type **est une** machine.
Usine → Machine (de 0 à plusieurs)	L'usine **manipule** des machines de tous types par le type commun, sans connaître leur type réel.
Usine, Machine → Ordre	L'usine confie des ordres ; la machine exécute l'ordre reçu, et l'ordre retient ses heures faites.
Demo, Menu → Usine	Les classes de test et de menu **utilisent** l'usine.